Instituto Politécnico do Cávado e do Ave
Escola Superior de Tecnologia

Engenharia Eletrotécnica e de Computadores

ROBÓTICA

TRABALHO PRÁTICO 1

Classificação de Sinais de Trânsito em Vídeo com MATLAB

Trabalho realizado por:

Tiago Oliveira nº 25547
Ivo Azevedo nº 15526

Novembro de 2025

Resumo

Neste relatório iremos abordar a realização do trabalho prático da unidade curricular de Robótica. O trabalho consistiu no desenvolvimento de um sistema de classificação de sinais de trânsito em tempo real a partir de uma *webcam*, utilizando o programa *MATLAB*.

Neste trabalho, desenvolvemos um sistema de visão computacional, onde primeiramente definimos os métodos de deteção e classificação, de seguida implementamos algoritmos de aquisição em tempo real, segmentação por cor e extração de características.

Seguidamente, prosseguimos para o desenvolvimento de uma interface gráfica (GUI) que permite a aquisição do vídeo em tempo real, o ajuste dinâmico de parâmetros de cores, a validação dos resultados obtidos e um histórico dos mesmos, que permitiu os cálculos de métricas.

No final da implementação, foram retiradas conclusões sobre o desempenho do sistema desenvolvido em diferentes cenários.

Índice

Introdução 5
1. Descrição do método desenvolvido 6
1.1- Aquisição e Pré-processamento 6
1.2- Segmentação por Cor 6
1.3- Pós-processamento da Máscara 6
1.4- Extração de Características 6
1.5- Classificação 7
1.6- Integração na Interface (GUI) 7
2. Diagrama de blocos do sistema 9
3. Explicação da escolha de parâmetros e métodos 11
3.1- Espaço de Cor (HSV vs RGB) 11
3.2- Filtragem Morfológica 11
3.3- *Thresholds* Dinâmicos 11
3.4- Critérios de Classificação 12
4. Resultados experimentais e análise de desempenho 13
4.1. Metodologia de Teste 13
4.2. Análise Quantitativa 14
4.3. Análise de Erros 15
5. Possíveis melhorias 15
Conclusão 16

Índice de figuras

Figura 1 – HomePanel 7
Figura 2 – DevRedPanel (esquerda) DevYellowPanel (direita) 8
Figura 3 - DevBluePanel 8
Figura 4 - HistoryPanel 8
Figura 5 - Espectro de cores RGB (esquerda) e HSV (direita) 11
Figura 6 - Calibração dinâmica das cores 12
Figura 7 - Painel do histórico com dados 14

Introdução

O presente trabalho foi desenvolvido na unidade curricular de Robótica. O principal objetivo consistiu no desenvolvimento de um sistema de classificação de sinais de trânsito em tempo real a partir de uma webcam, utilizando o programa *App Designer* do *MATLAB*.

Este trabalho é constituído por duas componentes principais, sendo a primeira focada no processamento de imagem e a segunda na interface gráfica (GUI) do sistema.

O processamento de imagem, consistiu na identificação de características relevantes de cada sinal, tais como:

- Cores;
- Formas.

Com a identificação dessas características foi possível implementar a lógica de classificação dos sinais.

Para a interface gráfica, foi desenvolvida uma aplicação que permite iniciar, pausar e parar a aquisição de vídeo em tempo real, ajustar *thresholds*, classificar os sinais em tempo real, verificar manualmente a identificação dos sinais e gravar os resultados num ficheiro para posterior análise de métricas de desempenho.

Para a realização de um trabalho mais rigoroso foi criada uma interface gráfica com o intuito de uma verificação de parâmetros, tais como:

- Quantidade de blobs;
- Circularidades;
- Centroids;
- Máscaras;
- Quantidade de pixéis de uma cor.

Posteriormente foram analisadas métricas de desempenho que foram obtidas através dos resultados guardados. As métricas estudadas foram:

- Acerto;
- Precisão;
- Sensibilidade *(Recall)*.

Por fim, foram retiradas conclusões sobre o desempenho do sistema desenvolvido em diferentes cenários e como os conceitos aprendidos nesta unidade curricular foram aplicados na elaboração deste trabalho.

1. Descrição do método desenvolvido

O método implementado para a classificação de sinais de trânsito passa por um processamento sequencial, dividido nas seguintes etapas fundamentais abaixo.

1.1- Aquisição e Pré-processamento

A aquisição de imagem é realizada através da *webcam* do computador, com o auxílio da Toolbox do *Matlab Image Acquisition*. O sistema captura *frames* no formato RGB com uma taxa de atualização de 20 fps, controlada por um *timer* (*TimerData*) que chama a função de processamento ao fim de cada 0,05 segundos. No pré-processamento, cada *frame* capturado é convertido do espaço de cor RGB para HSV.

1.2- Segmentação por Cor

A segmentação por cor é efetuada através de *thresholds* aplicados à imagem capturada pela *webcam*. A partir da interface é possível definir esses *thresholds* dinamicamente para as cores: vermelho, azul e amarelo, utilizando a função *ginput* ou introduzindo manualmente os valores. Já as cores preto e branco são *thresholds* fixos.

1.3- Pós-processamento da Máscara

A máscara binária resultante é submetida a uma sequência de operações morfológicas e de análise de *blobs*. Inicialmente, recorremos às funções *bwlabel* e *regionprops* para identificar objetos e extrair a sua geometria. Posteriormente, a máscara é processada com a função *imfill*, seguida de operações de *imerode*, *imclose* e *imopen*, obtendo-se assim a forma final do sinal para classificação.

1.4- Extração de Características

Após a identificação da cor predominante, extraímos características determinantes para a classificação de cada sinal. A circularidade, obtida pela função *regionprops*, permite distinguir formas geométricas como círculos e triângulos, enquanto a análise do conteúdo interno verifica a quantidade de objetos. Com a *Bounding Box* isolamos especificamente a largura e altura do sinal, permitindo verificar se o sinal é um quadrado, enquanto a propriedade *Centroid* é aplicada para comparar a posição horizontal do centro do objeto interno face ao centro do sinal, distinguindo assim a direção obrigatória.

1.5- Classificação

A classificação do sinal segue uma sequência de decisões baseadas nas características extraídas ao longo da análise do mesmo. A primeira verificação consiste na deteção de uma densidade significativa de pixéis da mesma cor (vermelho, azul ou amarelo) na imagem capturada.

Caso se confirme a predominância de uma cor, o sistema isola o objeto com maior área e analisa a sua forma geométrica. Através dos valores de circularidade, é possível fazer a distinção de objetos circulares, triangulares e quadrados. Após a identificação da forma geométrica, segue-se a análise ao interior do sinal através de submáscaras, permitindo identificar objetos e/ou cores internas para finalizar a classificação do sinal.

1.6- Integração na Interface (GUI)

A interface gráfica, desenvolvida no App Designer, encontra-se estruturada em diferentes painéis, que são acedidos através do botão “Menu”, permitindo assim separar o principal painel do sistema das ferramentas de diagnóstico e análise de dados.

Painel Principal (HomePanel): Neste painel encontram-se as principais funcionalidades do sistema. Assim que o utilizador carregue no botão “*Connect*”, é apresentado o vídeo em tempo real (imagem original). Depois de feita a calibração de todas as cores, ao premir o botão “*Start Processing*” é apresentado o vídeo em tempo real com a imagem processada. Neste painel também é possível verificar o resultado da classificação do sinal. Inclui campos para o ajuste manual dos *thresholds* e a interface de validação, onde o utilizador classifica a resposta do sistema como "Correta" ou "Incorreta". Esta validação atualiza automaticamente o histórico dos sinais processados e os cálculos de métricas.

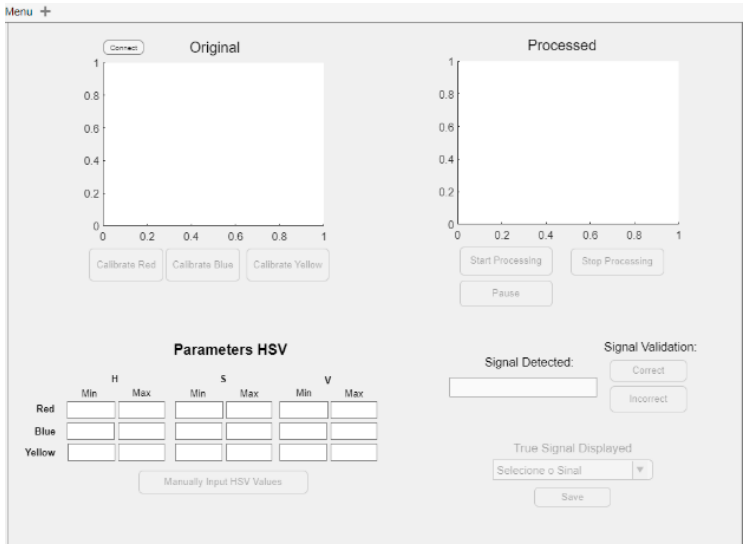


Figura 1 – HomePanel

Painéis de Análise (DevRedPanel, DevBluePanel e DevYellowPanel): Estes painéis, numa visão geral, permite ter acesso a valores e *plots* que nos permitem fazer *debugs* e visualizar passos intermédios do algoritmo. Para tal, foram criados painéis específicos para cada cor (Vermelho, Azul e Amarelo).

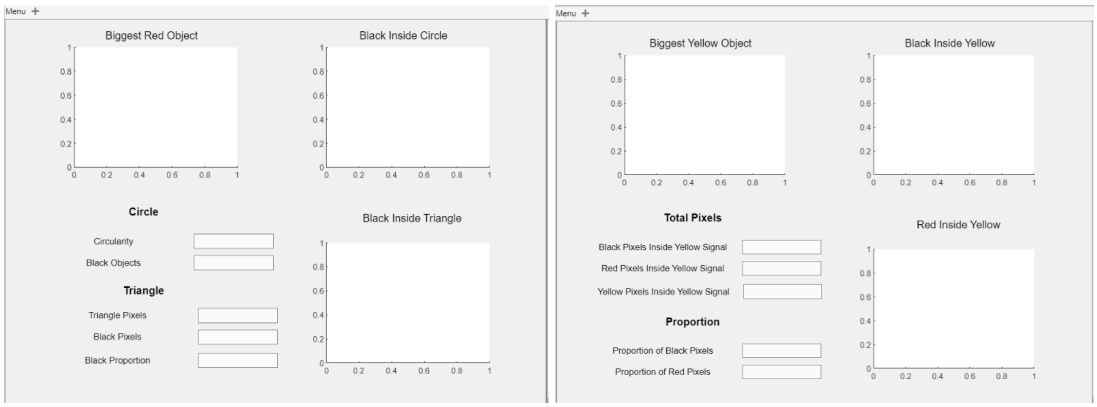


Figura 2 – DevRedPanel (esquerda) DevYellowPanel (direita)

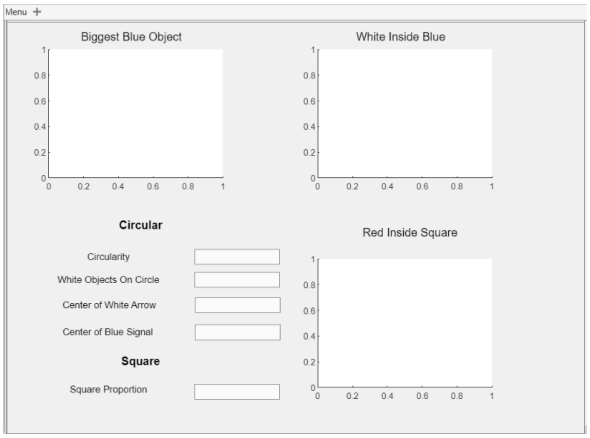


Figura 3 - DevBluePanel

Painel de Histórico e Métricas (HistoryPanel): Dedicado à monitorização do desempenho do sistema criado, este painel exibe uma tabela com o registo histórico de todas as validações manuais efetuadas juntamente com os *thresholds* utilizados e a data e hora, um gráfico de barras que compara a percentagem de classificações corretas e erradas, um gráfico de linhas e mostradores numéricos que apresentam as métricas calculadas.

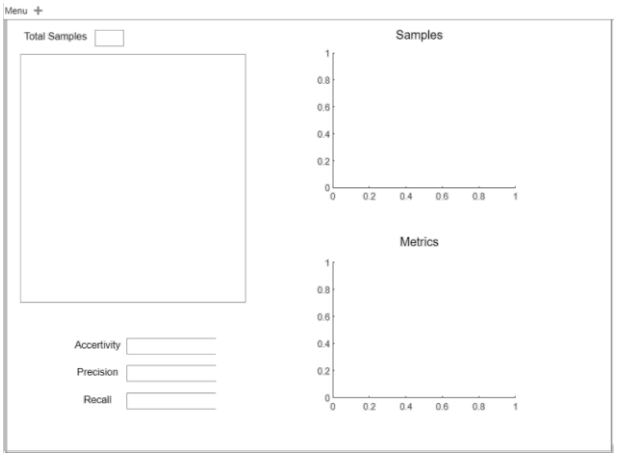
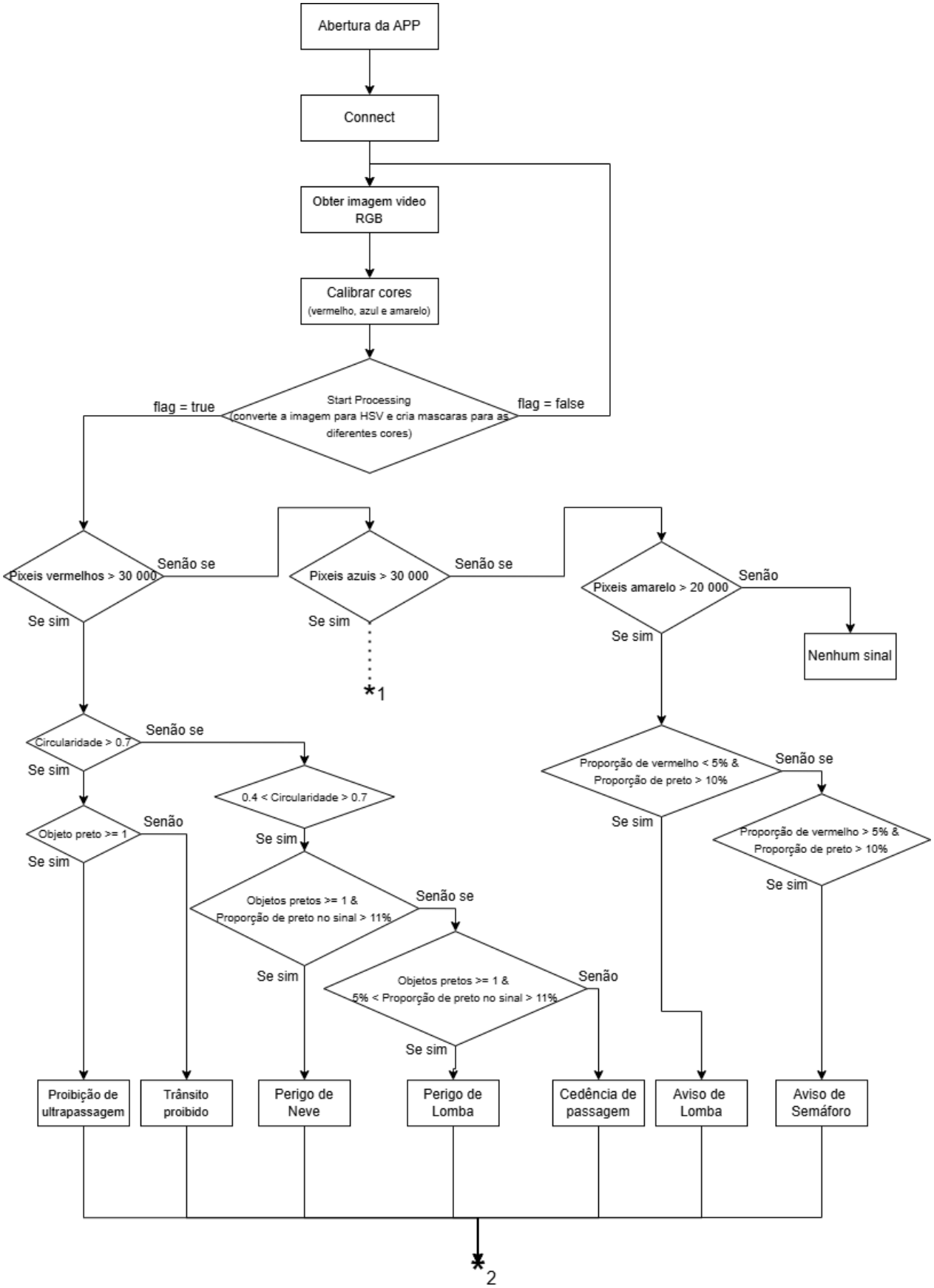
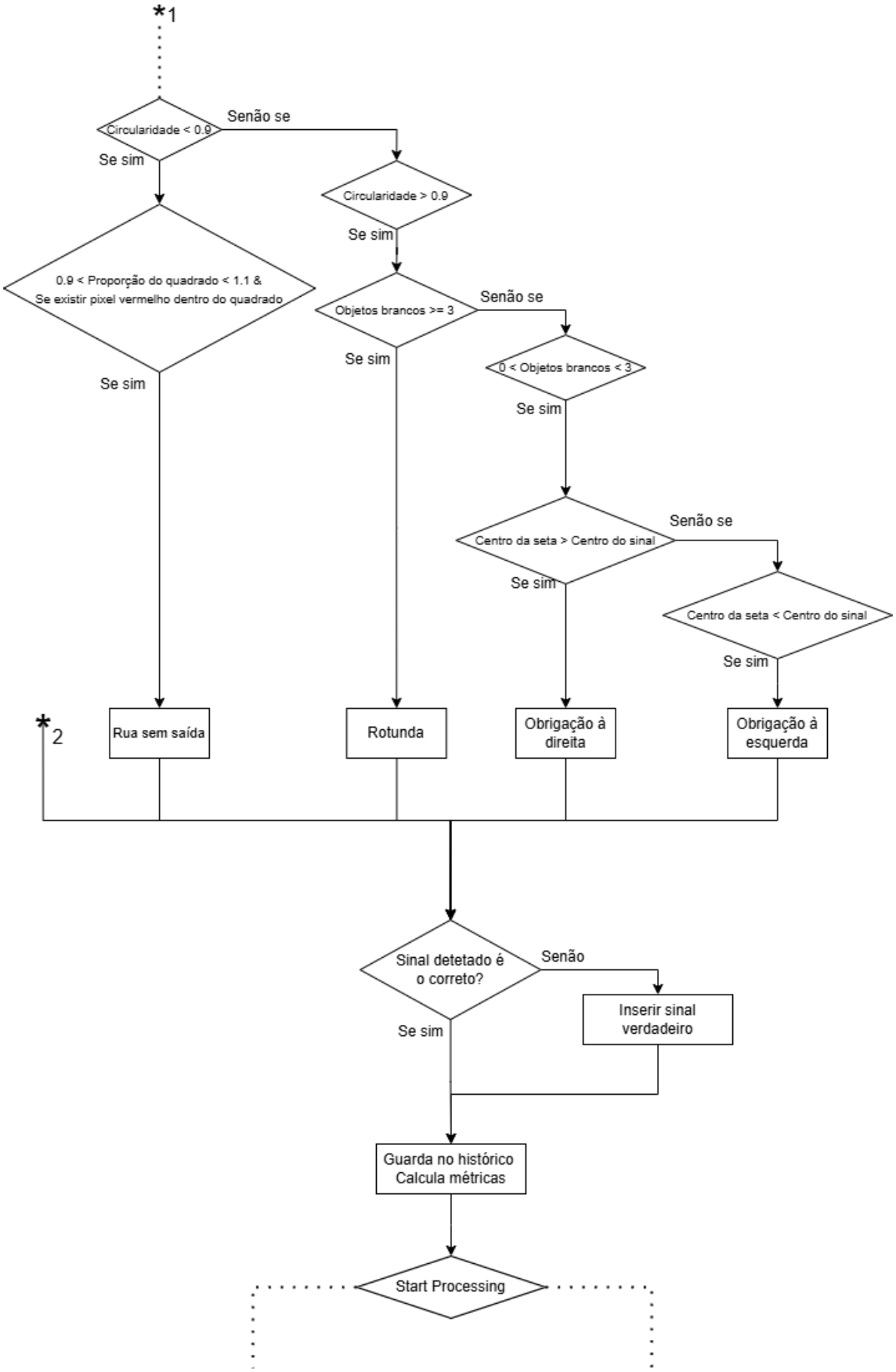


Figura 4 - HistoryPanel

2. Diagrama de blocos do sistema




*1
Circularidade < 0.9
Senão se
Se sim
Circularidade > 0.9
Se sim
0.9 < Proporção do quadrado < 1.1 &
Se existir pixel vermelho dentro do quadrado
Objetos brancos >= 3
Senão se
Se sim
0 < Objetos brancos < 3
Se sim
Se sim
Centro da seta > Centro do sinal
Senão se
Se sim
Centro da seta < Centro do sinal
Se sim
*2
Rua sem saída
Rotunda
Obrigação à direita
Obrigação à esquerda
Sinal detetado é o correto?
Senão
Inserir sinal verdadeiro
Se sim
Guarda no histórico
Calcula métricas
Start Processing

3. Explicação da escolha de parâmetros e métodos

3.1- Espaço de Cor (HSV vs RGB)

A escolha do espaço de cor HSV em relação ao RGB foi uma necessidade técnica imposta pelas condições não controladas de iluminação. No espaço de cores RGB, a informação de luminosidade está acoplada à informação de cor, o que significa que uma simples sombra altera os valores dos três canais (R, G, B), destabilizando a segmentação.

Ao converter para HSV, isolámos a cor no canal *Hue*, tornando o algoritmo robusto a variações de luz. Adicionalmente, a equalização do histograma no canal *Value* foi implementada para normalizar o brilho; desta forma, é possível compensar a fraca gama dinâmica da *webcam* e tornar a segmentação por cor mais robusta.

Figura 5 - Espectro de cores RGB (esquerda) e HSV (direita)

3.2- Filtragem Morfológica

A aplicação de filtros morfológicos serviu para resolver dois problemas distintos gerados pela binarização da imagem. Utilizámos a erosão e a abertura para eliminar o "ruído de sal", que podia ser detetado como objetos. Já para garantir a integridade dos sinais e preencher falhas internas usámos a dilatação e o fecho.

3.3- *Thresholds* Dinâmicos

Inicialmente, utilizámos valores de *threshold* fixos para criar uma base lógica de segmentação, mas a instabilidade da iluminação ambiente impôs a rápida implementação de limites dinâmicos. Constatámos que, apesar da equalização do histograma ajudar a normalizar o brilho, esta era insuficiente para garantir a fiabilidade de valores fixos perante variações de luz mais agressivas.

Para tornar a seleção dos valores de *threshold* dinâmicos, implementamos dois métodos na interface gráfica: a amostragem direta via função *ginput* (onde o utilizador seleciona 5 pontos da cor a calibrar, na imagem) e o ajuste fino manual dos canais HSV. Esta abordagem ajuda a que o sistema tenha um melhor desempenho em diferentes cenários sem a necessidade de alterar o código-fonte.

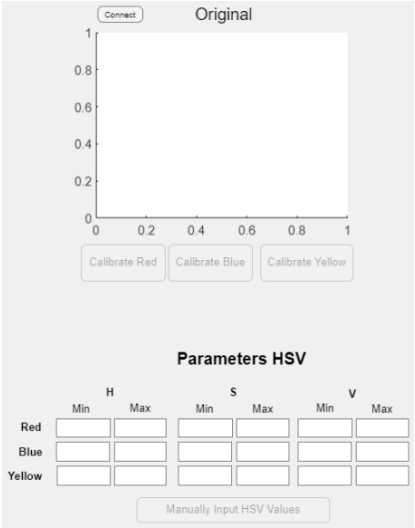


Figura 6 - Calibração dinâmica das cores

Também é importante salientar que os valores extraídos por esta amostragem não são assumidos como limites absolutos. Para garantir uma segmentação mais correta, o algoritmo aplica automaticamente uma margem de tolerância aos dados recolhidos: expande-se o intervalo do canal *Hue* (±0.08) e reduz-se o limite mínimo da *Saturation* e do *Value* (-0.15). Esta margem de segurança previne a exclusão de pixéis com ligeiras diferenças de cor que, embora pertencentes ao sinal, não foram captados nos pontos exatos da calibração.

3.4- Critérios de Classificação

Para classificar o sinal, começamos por verificar a cor predominante através da soma total de pixéis na imagem. Identificada a cor, utilizamos a propriedade área para isolar apenas o maior objeto, garantindo assim que o sistema analisa o sinal efetivo e descarta o ruído envolvente.

Para a definição da forma geométrica, optámos pela circularidade, pois permite distinguir círculos de triângulos e de quadrados. Seguidamente, analisamos o conteúdo interno (número de objetos e proporções) para refinar a classificação.

Por fim, recorremos a propriedades específicas para resolver ambiguidades: no caso do sinal quadrado, utilizámos a *Bounding Box* para calcular a proporção entre altura e largura, enquanto o *Centroid* foi aplicado para determinar a orientação das setas através da comparação de posições.

4. Resultados experimentais e análise de desempenho

Para garantir uma avaliação rigorosa, implementamos um sistema de histórico que regista por cada amostra os seguintes dados:

- Classificação atribuída pelo sistema;
- Verificação manual do utilizador;
- Sinal real (sinal mostrado ao sistema);
- Valores de *threshold*;
- Hora e data da classificação.

A recolha destes dados permite o cálculo automático das métricas fundamentais de desempenho: Taxa de Acerto (*Accuracy*), Precisão (*Precision*) e *Recall*.

4.1. Metodologia de Teste

De forma a possibilitar a realização de testes precisos, implementou-se na interface gráfica um botão de Pausa. Esta funcionalidade permite "congelar" o processamento de imagem em tempo real, facilitando a análise detalhada dos valores de *debug* e a validação manual do sinal detetado.

O processo de validação segue a seguinte lógica:

- **Se a deteção for correta:** O utilizador prime o botão **"***Correct***"**, guardando o registo no ficheiro *History.mat* e atualizando instantaneamente as métricas.
- **Se a deteção falhar:** É solicitado ao utilizador que selecione o sinal real numa lista. Ao premir o botão **"***Save***"**, o sistema regista o erro e recalcula o desempenho.

Com base nesta interação, o algoritmo categoriza automaticamente cada amostra numa classe lógica, que servirá para a análise quantitativa:

- **Verdadeiro Positivo (TP):** O sistema detetou um sinal e o utilizador validou a classificação como "Correta".
- **Falso Positivo (FP):** O sistema detetou um sinal, mas o utilizador marcou a classificação como incorreta.
- **Falso Negativo (FN):** O sistema indicou "Nenhum" sinal, mas o utilizador indicou que existia um sinal.
- **Verdadeiro Negativo (TN):** O sistema não detetou sinal e confirmou-se a sua inexistência.

4.2. Análise Quantitativa

Para efetuar uma avaliação do desempenho do sistema, recorremos aos dados categorizados durante a fase de testes (TP, FP, TN, FN). A partir destas variáveis foi nos possível calcular indicadores estatísticos que nos ajudam a analisar a fiabilidade do algoritmo desenvolvido.

A primeira métrica analisada foi a **taxa de acerto** (*Accuracy*). Esta métrica representa a percentagem de sucesso do sistema. Este valor engloba tanto a capacidade de identificar corretamente os sinais como a correta identificação da ausência dos mesmos, sendo calculada da seguinte forma:

$$Accuracy = \frac{TP + TN}{Total\ Amostras} * 100\ (\%)$$

Para avaliar a fiabilidade das classificações, calculamos a **taxa de precisão** (*Precision*). Este indicador mostra-nos a probabilidade de uma classificação do sistema ser acertada, penalizando os casos em que o sistema identifica falsos positivos, conforme a fórmula abaixo:

$$Precision = \frac{TP}{TP + FP} * 100\ (\%)$$

Por último, calculamos o ***Recall***, este diz-nos o quão capaz o sistema é de encontrar todos os sinais. Quanto maior for este valor, menor é a probabilidade de o algoritmo não conseguir identificar os sinais. O *Recall* é calculado da seguinte forma:

$$Recall = \frac{TP}{TP + FN} * 100\ (\%)$$

A figura abaixo ilustra o painel do histórico num cenário de testes, onde foram introduzidas classificações variadas para validar o correto registo na tabela e a resposta dinâmica dos cálculos métricos perante situações de acerto e erro.

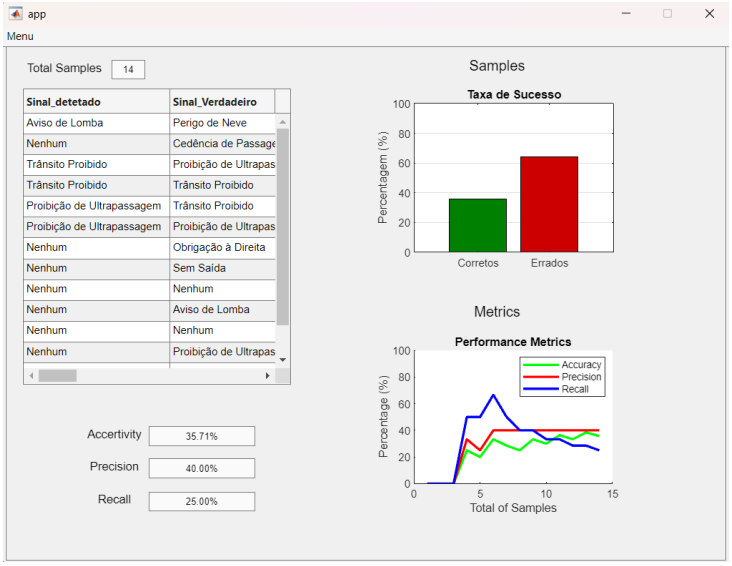


Figura 7 - Painel do histórico com dados

4.3. Análise de Erros

Durante o desenvolvimento deste trabalho verificámos que as limitações do sistema são fortemente condicionadas por fatores ambientais e geométricos. A principal fonte de erro foi a variação de luminosidade, onde zonas de luz ou sombras intensas comprometem a segmentação HSV, juntamente com a distância excessiva, que reduz a resolução do sinal dificultando o processo de extração de parâmetros para a classificação.

A orientação do sinal revelou-se importante, especialmente em sinais com conteúdo interno complexo, como no caso do sinal de "Aviso de Semáforo". Nestes casos, o desenho preto interior tende a fragmentar a máscara amarela. De forma a minimizar esse efeito, aplicámos um fecho morfológico (*imclose*) que ajuda a reconectar as áreas separadas, mas verificámos que esta técnica é insuficiente quando o sinal não se encontra totalmente frontal à câmara. Nessas situações, a combinação entre a fragmentação da máscara e a distorção de perspetiva impede que o sistema reconheça a geometria do sinal, resultando numa falha de deteção.

5. Possíveis melhorias

Apesar dos resultados satisfatórios obtidos com a abordagem atual, verificamos que existem algumas limitações no sistema que poderiam ser superadas a partir da implementação de técnicas adicionais de processamento de imagem abordadas na unidade curricular, e assim aumentar a robustez do classificador.

Uma evolução significativa seria a substituição dos limites globais (atualmente com margem de tolerância fixa) por uma Limiarização Adaptativa. Métodos adaptativos, como os algoritmos de *Niblack* ou *Bernsen*, calculam o limiar para cada pixel com base na estatística da sua vizinhança local. Esta abordagem permitiria ao sistema lidar muito melhor com iluminação não uniforme (como sombras parciais no sinal), garantindo uma segmentação correta mesmo quando uma parte da imagem está mais escura que a outra.

Ao nível do pré-processamento, a aplicação de Filtros Espaciais antes da conversão de cor traria benefícios na redução de ruído. A utilização de um filtro Gaussiano ou de Mediana à imagem original permitiria suavizar o "grão" da imagem binária sem perder as arestas fundamentais dos sinais. Esta abordagem resultaria em máscaras binárias mais limpas, simplificando a filtragem atual baseada apenas na quantidade bruta de pixéis de uma determinada cor e permitindo uma validação da presença da cor muito mais precisa e menos dependente de valores fixos.

Por fim, a robustez da classificação geométrica poderia ser reforçada através de algoritmos de Deteção de Contornos, como o operador de *Canny* ou *Sobel*. A utilização destes operadores permitiria extrair as arestas do sinal de forma independente da sua cor interior, oferecendo um critério complementar para validar se a forma detetada é efetivamente um triângulo, círculo ou quadrado, especialmente em situações onde a deteção da cor não é eficaz.

Conclusão

A realização deste trabalho permitiu adquirir conhecimentos práticos fundamentais na área de processamento de imagem, essenciais para o desenvolvimento futuro de sistemas de visão computacional.

Foram aplicados conceitos teóricos de visão computacional e programação em *MATLAB* adquiridos em aula, que nos permitiram superar os desafios da classificação de sinais de trânsito em tempo real.

Para atingir os objetivos propostos para este trabalho foi necessário desenvolver um algoritmo de identificação de sinais e posteriormente a criação de uma interface gráfica de controlo, análise e validação.

Através da análise das métricas obtidas, é possível concluir que o sistema é funcional e capaz de classificar corretamente os sinais num ambiente com iluminação controlada e a distâncias curtas, embora revele sensibilidade a variações ambientais.

Durante a fase experimental, identificaram-se dificuldades técnicas relevantes, tais como:

- A instabilidade da segmentação por cor perante variações de luminosidade, o que obrigou à implementação de limites dinâmicos e margens de tolerância;
- A perda de resolução espacial com o aumento da distância, tornando a imagem do sinal demasiado pequena para uma análise fiável;
- A distorção de perspetiva quando o sinal não está frontal, afetando a classificação.

Apesar das dificuldades mencionadas acima, conseguimos concluir o trabalho com sucesso, obtendo uma aplicação que cumpre os requisitos de deteção e classificação, permitindo-nos perceber a importância da calibração de parâmetros e do pré-processamento para a fiabilidade de um sistema de visão.